

# Practical 3

## Practice Lab Assignment

CODETANTRA

Home Learn Anywhere

202501040138@mitaoe.ac.in Support Logout

3.1.1. Numpy Array Operations

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

**Input Format:**

- The first line contains two space-separated integers, `r` and `c`, representing the number of rows and the number of columns.
- The next `r` lines contain `c` space-separated integers representing the elements of the array, entered row by row.

**Output Format:**

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use `reshape()` function to reshape the input array with the specified number of rows and columns.

numpyarr...

1 import numpy as np

2

3 # write your code here...

4 rows,cols=map(int,input().split())

5 elements=[]

6 for \_ in range(rows):

7 elements.extend(map(int,input().split()))

8 array=np.array(elements).reshape(rows,cols)

Average time: 0.009 s, 8.75 ms | Maximum time: 0.016 s, 16.00 ms

2 out of 2 shown test case(s) passed | 10 out of 10 hidden test case(s) passed

Test case 1: 16 ms | Debug

Expected output: 3 4, 1 2 3 4, 5 6 7 8, 9 10 11 12, [[1,2,3,4]], [-5,-6,-7,-8]

Actual output: 3 4, 1 2 3 4, 5 6 7 8, 9 10 11 12, [[1,2,3,4]], [-5,-6,-7,-8]

Terminal | Test cases

< Prev | Reset | Submit | Next >

## Lab Assignment

CODETANTRA

Home Learn Anywhere

202501040138@mitaoe.ac.in Support Logout

3.2.1. Numpy: Matrix Operations

The given code takes two  $3 \times 3$  matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

**Task:**

You are required to compute and display the results of the following matrix operations:

- Addition (`matrix_a + matrix_b`)
- Subtraction (`matrix_a - matrix_b`)
- Element-wise Multiplication (`matrix_a * matrix_b`)
- Matrix Multiplication (`matrix_a . matrix_b`)
- Transpose of Matrix A

**Input Format:**

- The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

**Output Format:**

The program should display the results of the operations in the following order:

- The result of Addition.
- The result of Subtraction.
- The result of Element-wise Multiplication.

matrixOp...

1 import numpy as np

2

3 # Input matrices

4 print("Enter Matrix A:")

5 matrix\_a = np.array([list(map(int, input().split())) for i in range(3)])

6

7 print("Enter Matrix B:")

Average time: 0.018 s, 17.75 ms | Maximum time: 0.021 s, 21.00 ms

2 out of 2 shown test case(s) passed | 2 out of 2 hidden test case(s) passed

Test case 1: 10 ms | Debug

Expected output: Enter Matrix A:, 1 2 3, 4 5 6, 7 8 9, Enter Matrix B:, 1 1 1

Actual output: Enter Matrix A:, 1 2 3, 4 5 6, 7 8 9, Enter Matrix B:, 1 1 1

Terminal | Test cases

< Prev | Reset | Submit | Next >

CODETANTRA

Home Learn Anywhere

202501040138@mitaoe.ac.in Support Logout

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- Vertical Stacking:** Stack the two matrices vertically (one below the other).

**Input Format:**

- The program should first prompt the user to input two  $3 \times 3$  arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

**Output Format:**

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

stacking.py

1 import numpy as np

2

3 # Input matrices

4 print("Enter Array1:")

5 arr1 = np.array([list(map(int, input().split())) for i in range(3)])

6

7 print("Enter Array2:")

Average time: 0.019 s, 19.25 ms | Maximum time: 0.028 s, 28.00 ms

2 out of 2 shown test case(s) passed | 2 out of 2 hidden test case(s) passed

Test case 1: 16 ms | Debug

Expected output: Enter Array1:, 1 2 3, 4 5 6, 7 8 9, Enter Array2:, 4 5 6

Actual output: Enter Array1:, 1 2 3, 4 5 6, 7 8 9, Enter Array2:, 4 5 6

Terminal | Test cases

< Prev | Reset | Submit | Next >

CODETANTRAHomeLearn Anywhere202501040138@mitaoe.ac.inSupportLogout

3.2.3. Numpy: Custom Sequence Generation04:34

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using `numpy` based on these inputs and print the generated sequence.

**Input Format:**

- The user will input three integer values: start, stop, and step, each on a new line.

**Output Format:**

- The program should print the generated sequence based on the input values.

Sample Test Cases+

customS...

```
1 import numpy as np
2
3 # Take user input for the start, stop, and step of the sequence
4 start = int(input())
5 stop = int(input())
6 step = int(input())
7
8 # Generate the sequence using np.arange()
```

Average time0.009 s9.25 msMaximum time0.012 s12.00 ms

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 112 msDebug

Expected output3102  
[3 5 7 9]

Actual output3102  
[3 5 7 9]

Test case 28 ms

TerminalTest cases

< PrevResetSubmitNext >

CODETANTRAHomeLearn Anywhere202501040138@mitaoe.ac.inSupportLogout

3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Oper...07:25

You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

- Arithmetic Operations:**
  - Compute the element-wise sum, difference, and product of the two arrays.
- Statistical Operations:**
  - Calculate the mean, median, and standard deviation of array A.
- Bitwise Operations:**
  - Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex:  $A_1$  OR  $B_1$ ).

**Input Format:**

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

**Output Format:**

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Sample Test Cases+

different...

```
1 import numpy as np
2
3 def array_operations(A, B):
4
5     # Convert A and B to NumPy arrays
6     a=np.array(A)
7     b=np.array(B)
8
```

Average time0.008 s8.33 msMaximum time0.013 s13.00 ms

1 out of 1 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 113 msDebug

Expected output1 2 3 45 6 7 8  
Element-wise Sum: 6 8 10 12  
Element-wise Difference: -4 -4 -4 -4  
Element-wise Product: 5 12 21 32  
Mean of A: 2.5

Actual output1 2 3 45 6 7 8  
Element-wise Sum: 6 8 10 12  
Element-wise Difference: -4 -4 -4 -4  
Element-wise Product: 5 12 21 32  
Mean of A: 2.5

TerminalTest cases

< PrevResetSubmitNext >

CODETANTRAHomeLearn Anywhere202501040138@mitaoe.ac.inSupportLogout

3.2.5. Numpy: Copying and Viewing Arrays06:41

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

**Input Format:**

- A single line of space-separated integers.

**Output Format:**

- After modifying the view:

Original array after modifying view: <original\_array>  
View array: <view\_array>

- After modifying the copy:

Original array after modifying copy: <original\_array>  
Copy array: <copy\_array>

copyAnd...

```
1 import numpy as np
2
3 inputlist = list(map(int,input().split(" ")))
4
5 # Original array
6 original_array = np.array(inputlist)
7
8 # Create a view
```

Average time0.005 s5.50 msMaximum time0.007 s7.00 ms

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 17 msDebug

Expected output10 20 30 40 50 60 70 80  
Original array after modifying view: [99 20 30 40 50 60 70 80]  
View array: [99 20 30 40 50 60 70 80]  
Original array after modifying copy: [99 20 30 40 50 60 70 80]  
Copy array: [10 88 30 40 50 60 70 80]

Actual output10 20 30 40 50 60 70 80  
Original array after modifying view: [99 20 30 40 50 60 70 80]  
View array: [99 20 30 40 50 60 70 80]  
Original array after modifying copy: [99 20 30 40 50 60 70 80]  
Copy array: [10 88 30 40 50 60 70 80]

TerminalTest cases

< PrevResetSubmitNext >

CODETANTRA

HomeLearn Anywhere

202501040138@mitaoe.ac.inSupportLogout

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

The given code in the editor takes a single array, array1, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- search\_value: The value to search for in the array.
- count\_value: The value to count its occurrences in the array.
- broadcast\_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. **Searching:** Find the indices where search\_value appears in array1 and print these indices.
2. **Counting:** Count how many times count\_value appears in array1 and print the count.
3. **Broadcasting:** Add broadcast\_value to each element of array1 using broadcasting, and print the resulting array.
4. **Sorting:** Sort array1 in ascending order and print the sorted array.

**Input Format:**

1. A single line containing space-separated integers representing array1.
2. An integer search\_value represents the value to search for in the array.
3. An integer count\_value represents the value to count in the array.
4. An integer broadcast\_value represents the value to add to each element of the array.

arrayOpe...

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split())))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
```

Average time0.011 s11.50 ms

Maximum time0.014 s14.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 114 ms

Debug

Expected output

Actual output

Value to search: 1

Value to count: 2

Value to add: 2

[0 1 2]

3

Terminal

Test cases

< Prev

Reset

Submit

Next >

CODETANTRA

HomeLearn Anywhere

202501040138@mitaoe.ac.inSupportLogout

3.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.

Operatio...

```
1 import numpy as np
2
3 a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
4
5 # 1. Print all student details
6 print("All student Details:\n",a)
7
8 # 2. print total students
```

Average time0.016 s16.00 ms

Maximum time0.016 s16.00 ms

1 out of 1 shown test case(s) passed

Test case 116 ms

Debug

Expected output

Actual output

All student Details:

[[301. 67. 77. 88.]

[302. 78. 88. 77.]

[303. 45. 56. 89.]

[304. 88. 98. 45.]

[305. 78. 88. 99.]

Terminal

Test cases

< Prev

Reset

Submit

Next >